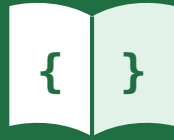

Bitirme Projesi — Kitapçı Platformu

Rehber

Spring Boot Edu



2026-09-25

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 Platforma Genel Bakış	2
2.2 Kurs Modüllerinin Buluştuğu Yer	2
3. Adım Adım Örnekler	3
3.1 Platformu Başlatmak	3
3.2 Sözleşmeler: gRPC ve Event'ler	4
3.3 Sipariş Vermek: gRPC Rezervasyonu ve Outbox	5
3.4 Dağıtık Kilit Altında Stok	6
3.5 Event'ler: Outbox Relay ve BookChanged	7
3.6 Arama Okuma Modeli ve Cache'i	8
3.7 Gateway: Route'lar, JWT ve Rate Limit	9
3.8 Her Şeyi Kapsayan Tek Trace	10
3.9 Dağıtık Bir Sistemi Test Etmek	11
3.10 Kubernetes'te Helm ile	12
4. Sık Yapılan Hatalar ve En İyi Uygulamalar	13
5. Özet	13
6. İleri Okuma	13

1. Öğrenme Hedefleri

Bitirme projesinin sonunda şunları yapabileceksiniz:

- dört Spring Boot servisi ve altı altyapı sisteminden oluşan bir sistemi tek komutla, ayrıca Kubernetes'te Helm ile çalıştırmak
- tek bir siparişi REST, gRPC, transactional outbox, Kafka, Elasticsearch ve Redis cache üzerinden takip etmek
- her teknolojinin neden orada durduğunu açıklamak (kararlar mimari dokümanında: ADR-1 ... ADR-8)
- dağıtık bir sistemi üç seviyede test etmek: fake'lerle tek servis, gerçek altyapıyla tek servis, tüm platform uçtan uca
- tek bir isteği Grafana'da tüm servisleri kapsayan tek bir trace olarak bulmak

Ön koşul: 00-24 arası modüller (bitirme projesi hepsini kullanır) · **Tahmini süre:** ödevlerle 8-12 saat

2. Kavramlar

2.1 Platforma Genel Bakış

Mimari dokümanı (docs/tr/mimari.md) sistemi ve kararlarını anlatır; bu rehber kodun içinden geçer. Kısaca:

Servis	Sahip olduğu	Konuştugu
gateway	tek giriş kapısı	tüm servisler (HTTP), Redis (rate limit)
order-service	siparişler (PostgreSQL)	katalog (gRPC), Kafka (outbox relay, bildirimler)
catalog-service	kitaplar ve stok (MongoDB)	Hazelcast (kilitler), Kafka (BookChanged)
search-service	arama okuma modeli (Elasticsearch)	Kafka (consumer), Redis (cache)

Her servisin kendi veritabanı vardır ve JWT'yi kendisi de kontrol eder; ilk kontrolü gateway yapar.

2.2 Kurs Modüllerinin Buluştuğu Yer

Konu	Modül	Bitirme projesinde
JPA, Flyway, transaction'lar	05, 06	siparişler, outbox, bildirimler
MongoDB	07	katalog, rezervasyonlar
Redis	08	arama cache'i, rate limiter
Hazelcast IMap kilidi	09	ISBN başına stok
Elasticsearch	10	arama okuma modeli
Kafka, outbox, idempotent consumer'lar	11	OrderPlaced, BookChanged
JWT resource server	12	gateway ve her servis
Testcontainers, fake'ler	14	her servisin testleri, uçtan uca test
OpenTelemetry, LGTM	15	sipariş başına tek trace
Docker, jlink	20	tüm servisler için tek Dockerfile
gRPC	21	stok rezervasyonu
Kubernetes, probe'lar	22	Helm chart
Spring Cloud Gateway	23	gateway

i Not

Bitirme projesi yeni kütüphane getirmez. Buradaki her şey bir modülde anlatıldı; yeni olan, parçaların birlikte nasıl çalıştığı — ve aralarında neyin ters gidebildiği.

3. Adım Adım Örnekler

3.1 Platformu Başlatmak

Tek komut dört imajı kaynak koddan derler (Maven Docker içinde çalışır) ve her şeyi başlatır:

```
cd capstone && docker compose up --build # ilk derleme: ~10 dakika, sonrasında
↪ saniyeler
```

Geliştirirken jar'ları makinede derlemek daha hızlıdır (capstone/up.sh bunu yapar ve compose'u JAR_SOURCE=host ile başlatır). Gateway 8080 portunu, Grafana 3000 portunu dinler; başka hiçbir port dışarı açılmaz. Akışı capstone/requests.http ile ya da curl ile deneyin:

```
TOKEN=$(curl -s -X POST localhost:8080/api/auth/token -H 'Content-Type:
↪ application/json' \
    -d '{"username":"ayse","password":"ayse-demo"}' | jq -r .accessToken)
curl -s "localhost:8080/api/search?q=kafka" # "sold": 0
curl -s -X POST localhost:8080/api/orders -H "Authorization: Bearer $TOKEN" \
    -H 'Content-Type: application/json' -d
    ↪ '{"lines":[{"isbn":"9781492078005","quantity":2}]}'
curl -s "localhost:8080/api/search?q=kafka" # kısa süre sonra:
↪ "sold": 2
```

★ İpucu

Token endpoint'i yalnızca gateway'in dev profilinde vardır (ADR-5). Demo parolaları application-dev.yaml'dan gelir ve ortam değişkenleriyle değiştirilebilir.

3.2 Sözleşmeler: gRPC ve Event'ler

Servisler birbirinin sınıflarını değil, yalnızca sözleşmeleri paylaşır. contracts kütüphanesi katalogun gRPC servisini ...

```
service StockService {
    // Reserves all lines or none. Idempotent per order_ref: a retry returns the first
    ↪ result.
    // NOT_FOUND: unknown ISBN · FAILED_PRECONDITION: not enough stock
    rpc ReserveStock (ReserveStockRequest) returns (ReserveStockResponse);
    // Gives the stock of a reservation back (when the order could not be saved).
    rpc ReleaseStock (ReleaseStockRequest) returns (ReleaseStockResponse);
}
```

... ve event'leri sade record'lar olarak tutar. Event'ler tip header'ı olmadan JSON olarak taşınır; böylece her consumer — hangi dilde yazılmış olursa olsun — onları okuyabilir:

```
public record OrderPlaced(String orderId, String customerId, List<Line> lines,
↪ BigDecimal total, Instant placedAt) {

    public static final String TOPIC = "bookstore.orders";

    public record Line(String isbn, String title, int quantity, BigDecimal unitPrice) {
    }
}
```

! Önemli

Bir sözleşme, çevresindeki koddan daha yavaş değişir. Alan ekleyin, ama asla yeniden

adlandırmayın ya da silmeyin; daha sonra deploy edilen bir servis eski mesajları da anlamak zorundadır.

3.3 Sipariş Vermek: gRPC Rezervasyonu ve Outbox

Sipariş servisi önce stoğu ayırır, sonra kaydeder. Rezervasyonun anahtarı siparişin ID'sidir; tekrarlanan bir çağrı yeni bir şey ayırmaz:

```
OrderResponse place(String customerId, PlaceOrderRequest request) {
    UUID id = UUID.randomUUID(); // also the idempotency key of
    ↪ the reservation
    Map<String, Integer> quantities = request.lines().stream().collect(Collectors.toMap(
        PlaceOrderRequest.Line::isbn, PlaceOrderRequest.Line::quantity, Integer::sum,
    ↪ LinkedHashMap::new));

    // the remote call runs OUTSIDE the transaction: no database connection is held while
    ↪ we wait
    List<ReservedBook> reserved = stock.reserve(id.toString(), quantities);
    try {
        return transactions.execute(status -> save(id, customerId, reserved));
    } catch (RuntimeException e) {
        releaseAfterFailure(id, e); // compensation: the catalog
        ↪ gets its copies back
        throw e;
    }
}
```

Üç ayrıntı önemlidir:

- gRPC çağrısı transaction'ın **dışında** yapılır. Başka bir servis çalışırken asla bir veritabanı bağlantısı tutulmaz.
- Sipariş ve OrderPlaced event'i **tek** bir transaction'da kaydedilir (ADR-3). Birinin diğeri olmadan var olduğu bir an yoktur.
- Kayıt başarısız olursa rezervasyon geri verilir — dağıtık transaction olmadığı için bir telafi (compensation) adımı.

Stok servisinin istemcisi her çağrıya bir deadline koyar ve müşterinin JWT'sini iletir (katalog da onu kontrol eder):

```
StockClient(StockServiceGrpc.StockServiceBlockingStub stock, CatalogProperties catalog)
↪ {
    // the interceptor asks for the token on every call: it is the token of the current
    ↪ request
    this.stock = stock.withInterceptors(new BearerTokenAuthenticationInterceptor(
        (Supplier<String>) StockClient::currentToken));
}
```

```
this.deadline = catalog.deadline();
}
```

gRPC durum kodları OrderErrors'ta HTTP cevaplarına dönüşür: FAILED_PRECONDITION → 409, NOT_FOUND → 422, UNAVAILABLE/DEADLINE_EXCEEDED → 503.

3.4 Dağıtık Kilit Altında Stok

İki katalog örneği yan yana çalışır (Kubernetes'te iki replika). Kilit olmadan ikisi de son kopyayı satabilirdi. Katalog siparişteki her ISBN'i Hazelcast'te, her zaman aynı (sıralı) düzende kilitler; böylece iki sipariş birbirini sonsuza kadar bekleyemez:

```
public Reservation reserve(String orderRef, Map<String, Integer> quantities) {
    var existing = reservations.findById(orderRef);
    if (existing.isPresent()) {
        return existing.get(); // idempotent: a retry
        ↪ of the same order
    }
    Map<String, Integer> sorted = new TreeMap<>(quantities); // always the same
    ↪ lock order
    List<String> locked = new ArrayList<>();
    try {
        sorted.keySet().forEach(isbn -> locked.add(lock(isbn)));
        List<Book> selected = sorted.entrySet().stream().map(entry ->
            ↪ available(entry.getKey(), entry.getValue())).toList();
        List<Reservation.Line> lines = new ArrayList<>();
        for (Book book : selected) {
            int quantity = sorted.get(book.isbn());
            books.save(book.withStock(book.stock() - quantity));
            lines.add(new Reservation.Line(book.isbn(), book.title(), quantity,
            ↪ book.price()));
        }
        return reservations.save(new Reservation(orderRef, lines,
            ↪ Reservation.Status.RESERVED));
    } finally {
        locked.forEach(locks::unlock);
    }
}
```

StockServiceTest bunu kanıtlar: beş kopyası olan bir kitap için on paralel sipariş — tam olarak beşi başarılı olur.

! Uyarı

Kilidin bir kira süresi (lease) vardır (10 sn). Çöken bir örnek kilitlerini kendiliğinden

birakır — ama kira süresinden uzun süren bir rezervasyon artık korunmaz. Kilitli bölümü kısa tutun: içinde uzak çağrı yapmayın.

3.5 Event’ler: Outbox Relay ve BookChanged

Zamanlanmış bir relay outbox satırlarını yayınlar ve gönderildi olarak işaretler. FOR UPDATE SKIP LOCKED sayesinde birden çok sipariş servisi örneği, bir satırı iki kez göndermeden paralel çalışabilir:

```
@Scheduled(fixedDelayString = "${bookstore.outbox.relay-interval}")
@Transactional
public void publishPending() throws ExecutionException, InterruptedException,
    ↪ TimeoutException {
    List<OutboxEvent> pending = events.lockNextToPublish();           // other
    ↪ instances skip these rows
    for (OutboxEvent event : pending) {
        kafka.send(event.getTopic(), event.getEventKey(), event.getPayload())
            .get(10, TimeUnit.SECONDS);                             // wait for the
        ↪ broker's acknowledgement
        event.markPublished(clock.instant());                       // saved at commit
    ↪ (dirty checking)
    }
}
}
```

Katalog, bir kitabın **her** kaydından sonra Spring Data MongoDB’nin yaşam döngüsü event’iyle BookChanged yayınlar. Admin değişiklikleri, başlangıç verisi ve stok rezervasyonları hep repository’den geçer; hiçbiri unutulamaz:

```
@Override
public void onAfterSave(AfterSaveEvent<Book> event) {
    Book book = event.getSource();
    var changed = new BookChanged(book.isbn(), book.title(), book.authors(),
    ↪ book.description(), book.price(),
        book.stock());
    kafka.send(BookChanged.TOPIC, book.isbn(), changed)
        .whenComplete((result, failure) -> {
            if (failure != null) {
                log.warn("BookChanged for {} not published: {}", book.isbn(),
    ↪ failure.getMessage());
            }
        });
}
```

i Not

Katalogun outbox'ı yoktur — bilinçli bir ödünleşim. Kafka kapalıysa arama indeksi, kitap tekrar değişene kadar değişikliği kaçırır. Siparişler için bu kabul edilemezdi; bir arama indeksi için katlanılabilir. İki yaklaşımı karşılaştırın.

3.6 Arama Okuma Modeli ve Cache'i

Arama servisinin kendine ait verisi yoktur: indeksi yalnızca event'lerden oluşur (ADR-4). Satış sayımı tekrar gelen mesajlara dayanmalıdır — Kafka en az bir kez iletir —; bu yüzden Redis hangi siparişlerin sayıldığını hatırlar (SET NX) ve Elasticsearch sayıları bir script ile atomik olarak ekler:

```
@CacheEvict(cacheNames = "search", allEntries = true)
public void recordSale(OrderPlaced order) {
    String mark = "search:sale-counted:" + order.orderId();
    if (!Boolean.TRUE.equals(redis.opsForValue().setIfAbsent(mark, "1",
        ↪ REMEMBER_SALES))) {
        return; // counted before
    }
    try {
        order.lines().forEach(line ->
        ↪ elasticsearch.update(UpdateQuery.builder(line.isbn())
            .withScript("ctx._source.sold += params.quantity") // atomic inside
            ↪ Elasticsearch
            .withParams(Map.of("quantity", line.quantity()))
            .withRefreshPolicy(RefreshPolicy.IMMEDIATE)
            .build(), BOOKS));
    } catch (RuntimeException e) {
        redis.delete(mark); // not counted: let the
        ↪ redelivery try again
        throw e;
    }
}
```

Arama sonuçları normalleştirilmiş sorgu başına Redis'te ön belleğe alınır. Her indeks değişikliği cache'i boşaltır (indeks metodlarında @CacheEvict(allEntries = true)); yeni bir satış hemen görünür:

```
@Cacheable(cacheNames = "search", key = "#query.strip().toLowerCase()") // "Java"
↪ and " java" share an entry
public SearchResult search(String query) {
    NativeQuery search = NativeQuery.builder()
        .withQuery(q -> q.multiMatch(match -> match
```

```

        .query(query)
        .fields("title^3", "authors^2", "description")      // a title
        ↪ match counts most
        .fuzziness("AUTO"))                                // "effective"
        ↪ still finds "effective"
        .withMaxResults(20)
        .build();
var hits = elasticsearch.search(search, BookDocument.class).stream()
        .map(SearchHit::getContent).map(BookHit::from).toList();
return new SearchResult(query, hits);
    }
}

```

3.7 Gateway: Route'lar, JWT ve Rate Limit

Gateway, path'e göre yönlendirir. Aynı rate limiter her route'u korur:

```

routes:
- id: orders
  uri: ${bookstore.services.orders}
  predicates:
    - Path=/api/orders/**
  filters:
    - &rate-limit
      name: RequestRateLimiter                # token bucket in Redis, shared by all
↪ gateway instances
    args:
      redis-rate-limiter.replenishRate: ${bookstore.rate-limit.replenish-rate}
      redis-rate-limiter.burstCapacity: ${bookstore.rate-limit.burst-capacity}
      key-resolver: "#{@customerOrIpKeyResolver}"
- id: catalog
  uri: ${bookstore.services.catalog}
  predicates:
    - Path=/api/catalog/**
  filters:
    - *rate-limit
- id: search
  uri: ${bookstore.services.search}
  predicates:
    - Path=/api/search/**
  filters:
    - *rate-limit

```

Bir istek kimin kovanını (bucket) kullanır? Giriş yapmış bir müşteri kendi kovanını — bir müşteri diğerlerini yavaşlatamasın diye; anonim bir istek ise IP adresinin kovanını:

```
@Bean
KeyResolver customerOrIpKeyResolver() {
    return exchange -> exchange.getPrincipal()
        .map(Principal::getName)
        .map(subject -> "customer:" + subject)
        .switchIfEmpty(Mono.fromSupplier(() -> "ip:" +
            Optional.ofNullable(exchange.getRequest().getRemoteAddress())
                .map(InetSocketAddress::getHostString).orElse("unknown"))));
}
```

Gateway'in security zinciri ilk kontroldür (token yoksa 401, kataloğu değiştirmeye çalışan müşteriye 403). Authorization header'ı değiştirilmeden iletilir ve her servis onu yeniden kontrol eder — derinlemesine savunma (ADR-5).

3.8 Her Şeyi Kapsayan Tek Trace

Her servis metrikleri ve trace'leri OTLP ile Grafana LGTM'ye aktarır. Kafka gözlemi trace bağlamını kayıt header'larında taşır:

```
tracing:
  sampling:
    probability: 1.0 # every request is traced (production: e.g.
    ↪ 0.1)
  observations:
    enable:
      tasks.scheduled: false # no trace for every relay run (twice a
      ↪ second)
  metrics:
    tags:
      application: ${spring.application.name}
  opentelemetry:
    tracing:
      export:
        otlp:
          endpoint: ${OTLP_ENDPOINT:http://localhost:4318}/v1/traces
  otlp:
    metrics:
      export:
        url: ${OTLP_ENDPOINT:http://localhost:4318}/v1/metrics
        step: 10s
```

Bir sipariş verin, Grafana'yı açın (<http://localhost:3000>) → Explore → Tempo ve gateway servisini arayın. POST /api/orders trace'i gateway'i, sipariş servisini, katalogdaki StockSer-

vice/ReserveStock gRPC çağrısını, bookstore.catalog send Kafka mesajını ve indeksini güncelleyen arama servisini gösterir — beş servis, tek trace.

i Not

OrderPlaced event'i yeni bir trace başlatır: outbox satırı trace bağlamını taşımaz ve relay kendi zamanlamasıyla çalışır. Bağlamı taşımak, ödevlerdeki ek görevlerden biridir.

3.9 Dağıtık Bir Sistemi Test Etmek

Bitirme projesi üç seviyede test edilir:

1. **Fake'lerle tek servis.** Sipariş servisinin testleri sahte bir StockService'i in-process başlatır (Spring gRPC'nin test transport'u). Fake kendisine sorulanları kaydeder; böylece bir test iletilen token'ı ya da geri verilen rezervasyonu kontrol edebilir:

```
@TestConfiguration(proxyBeanMethods = false)
class FakeCatalog {

    public static final FakeStockService STOCK = new FakeStockService();

    @Bean
    @GrpcService
    FakeStockService fakeStockService() {
        return STOCK;
    }

    @Bean
    @GlobalServerInterceptor
    FakeStockService.Recorder recordHeaders() {
        return new FakeStockService.Recorder(STOCK);
    }
}
```

2. **Gerçek altyapıyla tek servis.** Testcontainers, servisin gerçekten kullandığı ne varsa onu başlatır: PostgreSQL, MongoDB, Elasticsearch, Redis, Kafka ya da Hazelcast. Küçük bir TopicReader event'leri ham JSON olarak okur — başka bir servisin göreceği gibi.
3. **Tüm platform.** PlatformIT o anki build'in jar'larından dört imajı derler, compose.yaml'ı (LGTM olmadan) başlatır ve bir siparişi gateway üzerinden baştan sona geçirir:

```
static final ComposeContainer PLATFORM = new ComposeContainer(
    new File("../compose.yaml"), new File("compose.e2e.yaml"))
    .withBuild(true) // images from the jars of
    ↪ this build
    .withServices("gateway") // and everything it depends
    ↪ on (not LGTM)
    .withExposedService("gateway", 8080, Wait.forHttp("/actuator/health/readiness")
        .forStatusCode(200).withStartupTimeout(Duration.ofMinutes(10)));
```

★ İpucu

Hataların çoğu servislerin arasında yaşar, ama testlerin çoğu orada yaşamamalıdır. Uçtan uca testi ana akışla sınırlı tutun; ayrıntıları hızlı servis testlerinde test edin.

3.10 Kubernetes'te Helm ile

k8s/helm/bookstore Helm chart'ında dört servis için bir, geliştirme altyapısı için bir genel şablon vardır. Her servis küçük bir değer bloğudur:

```
catalog-service:
  image: springbootedu/capstone-catalog-service:latest
  replicas: 2 # two instances share the stock locks in
  ↪ Hazelcast (ADR-6)
  grpcPort: 9090
  env:
    MONGODB_URI: mongodb://mongo:27017/catalog
    HAZELCAST_ADDRESS: hazelcast:5701
  waitFor: [mongo:27017, hazelcast:5701, kafka:9092]
```

capstone/k8s/deploy-kind.sh imajları derler, kind'a yükler ve helm upgrade --install çalıştırır. İki katalog replikası Hazelcast kilitlerini paylaşır; sipariş servisi ikisine de headless bir Service üzerinden ulaşır ve gRPC çağrılarını round robin ile dağıtır:

```
@Bean
<T extends ManagedChannelBuilder<T>> GrpcChannelBuilderCustomizer<T>
  ↪ catalogRoundRobin() {
  return GrpcChannelBuilderCustomizer.matching("catalog", builder ->
    ↪ builder.defaultLoadBalancingPolicy("round_robin"));
  }
}
```

!! Dikkat

Kubernetes her Service için KAFKA_PORT=tcp://10.96... gibi değişkenler ekler. Kafka

imajı her KAFKA_* değişkenini yapılandırma olarak okur ve \${REDIS_PORT:6379} birden bir URL'ye çözülür. Chart her pod'da enableServiceLinks: false kullanır.

4. Sık Yapılan Hatalar ve En İyi Uygulamalar

- Bir veritabanı transaction'ı içinde başka bir servisi çağırmak. Önce uzak çağrı (ya da commit'ten sonra), sonra kısa bir transaction.
- Kaydettikten hemen sonra event yayınlamak (save() ardından kafkaTemplate.send()): ikisinden biri başarısız olabilir. Aynı transaction'da bir outbox satırı, onu bir relay yayınlarsın.
- Tam olarak bir kez iletim varsayan consumer'lar. Idempotent consumer'lar: primary key, ON CONFLICT DO NOTHING, bir SET NX işareti.
- Aynı veriyi "say, sonra hepsini kaydet" ile yükleyen iki replika: ikincisi duplicate key hatasıyla çöker. Doküman başına insert ve DuplicateKeyException'ı yok saymak (CatalogSeeder).
- Headless bir Service arkasında varsayılan pick-first politikasıyla bir gRPC istemcisi: her çağrı aynı pod'a gider. round_robin.
- application.yaml'da varsayılan değerli secret'lar. Varsayılsız \${BOOKSTORE_JWT_SECRET}; compose bir geliştirme değeri verir, Helm chart bir tane üretir.
- Her ayrıntı için bir uçtan uca test. Uçtan uca tek akış, ayrıntılar servis testlerinde.

5. Özet

- Bitirme projesi kursu birleştirir: dört servis, altı altyapı sistemi, tek komut (docker compose up --build) ve Kubernetes'te Helm ile aynı imajlar.
- Bir sipariş gRPC ile senkron olarak ayrılır ve bir outbox üzerinden asenkron olarak yayınlanır; arama indeksi event'lerden kurulan bir okuma modelidir.
- Dağıtık sistemler her yerde idempotency ister: sipariş ID'sine göre rezervasyon, sipariş ID'sine göre satış, primary key'e göre bildirim.
- Gözlemlenebilirlik parçaları birbirine bağlar: gateway'den arama indeksine tek trace.
- Üç seviyede test sistemi değiştirilebilir tutar: fake'ler, Testcontainers, uçtan uca tek akış.

6. İleri Okuma

- Bu platformun mimarisi ve kararları: docs/tr/mimari.md
- Transactional outbox deseni: <https://microservices.io/patterns/data/transactional-outbox.html>
- Spring gRPC referansı: <https://docs.spring.io/spring-grpc/reference/>

- Spring Cloud Gateway — RequestRateLimiter: <https://docs.spring.io/spring-cloud-gateway/reference/spring-cloud-gateway-server-webflux/gatewayfilter-factories/requestratelimiter-factory.html>
- Hazelcast — map kilitleme: <https://docs.hazelcast.com/hazelcast/5.5/data-structures/locking-maps>
- Helm dokümantasyonu: <https://helm.sh/docs/>